2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD) | 979-8-3315-1560-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCAD66269.2025.11240963

PAR-CIM: A Precise/Approximate Reconfigurable Digital CIM Macro with 0.35-4b Fractional Mixed-Bitwidth Quantization

Han Zhang*, Zhenyu Xue*, Wente Yi, Tianshuo Bai, Lehao Tan, Jingcheng Gu, Weijie Ding, Wang Kang, and Biao Pan†

*Equally Credited Authors. Beihang University, Beijing, China (†Contact Email: panbiao@buaa.edu.cn)

***Abstract*—Digital computing-in-memory (DCIM) enables efficient deep neural networks (DNNs) acceleration but faces limitations in resource overhead, energy efficiency, and architectural flexibility. Existing approximate or reconfigurable DCIM solutions tackle these issues partially without achieving a holistic balance. To address this, we propose PAR-CIM, a highly energy-efficient reconfigurable CIM macro that integrates precise and approximate paradigm. First, we introduce layer/gate-level approximate computation (LGAC) into the adder tree (AT) of the DCIM core, achieving full operation with only 0.35× the area of traditional implementations. Then, we develop a 0.35-4b fractional mixed-bitwidth quantization (FMBQ) algorithm, combining second-order Taylor sensitivity analysis with DoReFa-Net. This is complemented by a high-precision low-approximation (HPLA) mapping scheme to enhance energy efficiency. Additionally, a multi-bit reconfigurable computation mode (MBRM) strategy further improves architectural flexibility and enables the implementation of the proposed design. Under 40nm technology, PAR-CIM achieves 3048 TOPS/W at 1b/1b operations. With FMBQ, ResNet18 and our custom V-FuseMBA trained on CIFAR-10 achieve over 86.61% compression with accuracy loss under 0.74%, reaching classification accuracies of 93.67% and 92.86%, respectively.**

***Index Terms*—DCIM, Precise/Approximate Computation, Reconfigurable Architecture, Fractional Quantization**

I. Introduction

With the rapid advancement of deep neural networks (DNNs) and other artificial intelligence (AI) technologies, various network models are deployed on AI chips to tackle increasingly complex applications. However, traditional AI chips rely on the *Von Neumann* architecture, which separates storage and computation, creating substantial data transfer overhead—known as the *Memory Wall* [1]. To address this, digital computing-in-memory (DCIM) has emerged as a promising solution, offering acceleration for DNNs processing by integrating computation directly within memory [2].

DCIM enhances energy efficiency while providing high precision and robustness through digital logic. However, it still relies on multiplier arrays and adder trees (ATs), which consume significant area and power, leading to considerable resource overhead [3]. Furthermore, DCIM exhibits limited flexibility, making it challenging to support diverse computations [4]. Consequently, achieving an optimal balance among resource overhead, energy efficiency, and architectural flexibility remains a critical research target.

In recent years, approximate DCIM (ADCIM) and reconfigurable DCIM (RDCIM) have been proposed to overcome the limitations of traditional DCIM. As illustrated in Fig. 1(b),

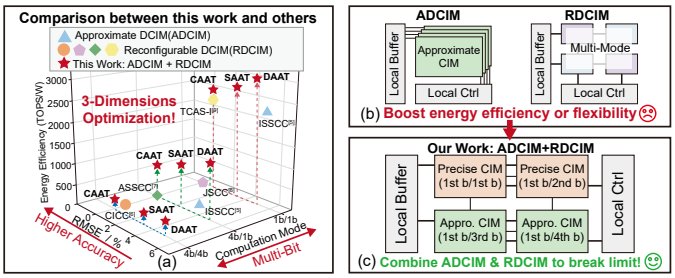


Fig. 1. Comparison between this work and others, (a) advantages in the 3D plot, (b) past works, and (c) our work.

ADCIM introduces minor computational errors to significantly optimize chip area and power consumption [5]. Meanwhile, RDCIM enhances architectural flexibility through techniques such as reconfigurable CIM arrays and multi-bit computation mode [6] [7] [8] [9]. However, Fig. 1(a) presents a three-dimensional analysis—energy efficiency, root mean square error (RMSE), and computation mode as the axes—highlighting that no previous DCIM reaches all three dimensions, underscoring the need for a more holistic approach.

Therefore, as shown in Fig. 1(c), we propose PAR-CIM based on ADCIM and RDCIM: a reconfigurable DCIM macro that integrates both precise and approximate computation, with 0.35-4b fractional mixed-bitwidth quantization (FMBQ). It achieves optimal performance across three dimensions through joint optimizations in circuit, algorithm, and architecture. The key contributions of this work are summarized as follows:

- **LGAC-AT**, an ATs design that integrates layer/gate-level approximate computation (LGAC) with three selectable options, balancing trade-offs among accuracy, power, and area to meet diverse application requirements.
- **FMBQ-HPLA**, a quantization algorithm named FMBQ based on second-order Taylor sensitivity analysis and DoReFa-Net [10], combined with a high-precision low-approximation (HPLA) scheme for efficient data storage.
- **MBRM**, a multi-bit reconfigurable computation mode (MBRM) strategy based on configurable dataflow, significantly enhances architectural flexibility and increases the parallelism of multiply-accumulate (MAC) computations.
- **Comprehensive experiments**, we evaluate PAR-CIM, achieving 3048 TOPS/W at 1b/1b operations. Using ResNet18 [11] and our custom Mamba-based [12] V-FuseMBA model, we achieve over 86.61% compression with accuracy loss below 0.74%, resulting in accuracies of 93.67% and 92.86% on CIFAR-10.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.

II. Background and Motivation

A. Approximate DCIM (ADCIM)

Circuit restructuring based on approximate computation paradigms has been demonstrated as a viable approach to reduce hardware overhead in CIM. Unlike analog CIM (ACIM), which has random errors from temperature and manufacturing variations [13], approximate computation uses deterministic errors introduced by designers. Therefore, combining approximate computation with ACIM makes it difficult to identify error sources. DCIM doesn't have this limitation [14], and its typical architecture is shown in Fig. 2(b). In recent years, several pioneering ADCIM designs have been published. Wang et al. proposed DIMC, in which the first two layers of ATs utilize AND/OR gates for approximate computation, followed by standard full adders and half adders in subsequent layers, achieving a 46.4%-56.4% area reduction per bit in the 1b/1b computation mode [5]. He et al. replaced multiplication with L_1 distance calculations, improving energy efficiency by over 3× compared to DCIM under the same technology [15]. As shown in Fig. 2(a), Guo et al. introduced an approximate 4:2 compressor-based AT utilizing NOR gates, reducing chip area and power by 63.10% and 58.85%, respectively [16]. Despite these significant breakthroughs, existing approaches still exhibit relatively high RMSE values and lack bitwidth scalability in computational architectures, substantially limiting ADCIM's full potential for DNN acceleration and hindering its widespread adoption in resource-constrained environments.

B. Reconfigurable DCIM (RDCIM)

RDCIM enhances architectural flexibility through dynamic array reconfiguration and inter-core interconnection frameworks, overcoming ADCIM's computational limits. The increasing depth of DNNs leads to a rapid growth in parameter storage and computation demands. DNN quantization effectively reduces these burdens, resulting in multi-precision network layers. Conventional solutions require equipping the entire network with multi-precision hardware support, which RDCIM aims to address. Thus, RDCIM has become a prominent research area. As illustrated in Fig. 2(c), Jia et al. proposed a flexible computation mapping method with scalable input-output tensors, allowing precision adjustment from INT1 to INT8 [17]. Tu et al. introduced ReCIM, which utilizes hierarchical and reconfigurable in-memory accumulators to flexibly support BF16/FP32 and INT8/INT16 in the same CIM macro [18]. Sharma et al. presented a 64Kb non-volatile ReRAM-based RDCIM macro, enabling full reconfigurability from 1b to 16b precision for both input and weight. This macro allows weight bit precision programming from 1b to 8b using a single macro, and from 1b to 16b using two parallel macros [19]. Despite significant progress, redundant reconfigurable logic introduces area and power overhead, while challenges in computational efficiency remain unresolved. Moreover, the neglect of network-level quantization limits layer-wise precision optimization, hindering the full potential of quantization and RDCIM for DNN acceleration.

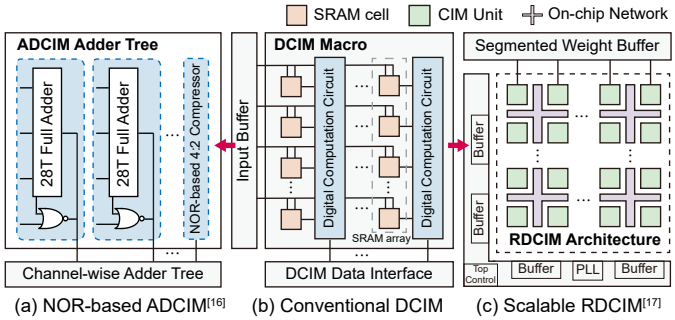


Fig. 2. Typical architectures: (a) NOR-based ADCIM, (b) conventional DCIM, and (c) scalable RDCIM.

C. Motivation

As previously mentioned, employing ADCIM or RDCIM independently struggles to address the challenges faced by traditional DCIM across multiple dimensions. To tackle this, we propose PAR-CIM—a reconfigurable DCIM macro that integrates precise and approximate computations, effectively combining the strengths of ADCIM and RDCIM to optimize hardware resource overhead, energy efficiency, and architectural flexibility. Furthermore, most existing studies overlook the varying quantization precision requirements of different network layers, often leading to low energy and area efficiency. In response to this, we present FMBQ, a novel fractional mixed-bitwidth quantization algorithm. Based on second-order Taylor sensitivity analysis and DoReFa-Net, this algorithm first performs a layer-wise sensitivity evaluation of the DNNs. Subsequently, it determines and applies an appropriate quantization bit for each layer, thereby significantly reducing the network's size while maintaining acceptable accuracy. Moreover, we have innovatively developed and successfully deployed the V-FuseMBA model, an image classification model based on an improved Mamba architecture. Its excellent performance on PAR-CIM demonstrates the application potential of our solution in emerging network models.

TABLE 1. Short Forms and Full Forms of Keywords.

Short Form	Full Form
SAAT	Single Approximate Adder Tree
DAAT	Double Approximate Adder Tree
CAAT	Carry-out Approximate Adder Tree
HP-Core	High-Precision Core
LA-Core	Low-Approximation Core
MMRC	Multi-Mode Reconfigurable Circuit
LGAC	Layer/Gate Approximate Computation
FMBQ	Fractional Mixed-Bitwidth Quantization
HPLA	High-Precision Low-Approximation
MBRM	Multi-Bit Reconfigurable Computation Mode

To better showcase our work, Tab. 1 presents the forms of frequently used keywords in the following text. Section III will delve into a detailed explanation of the three key features illustrated in Fig. 3. Subsequently, Section IV will present comprehensive evaluation results of the proposed PAR-CIM architecture. Finally, Section V will conclude our work with a concise summary of our findings and contributions.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.

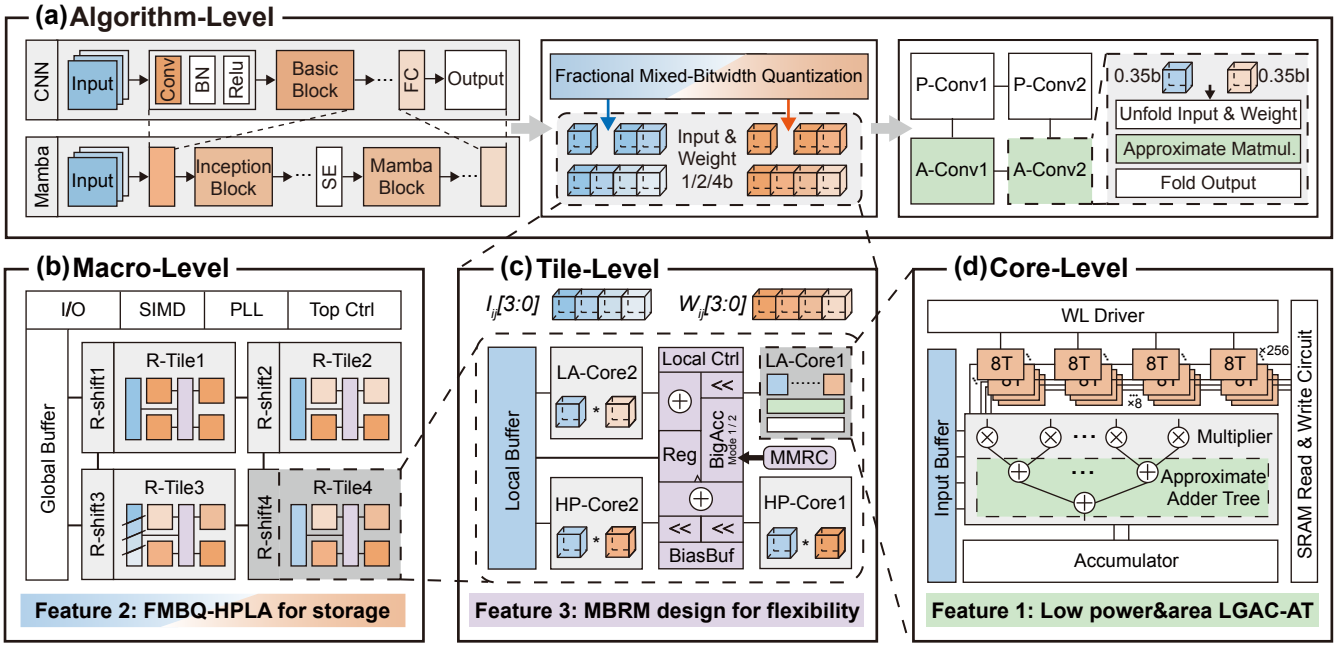


Fig. 3. Overview of PAR-CIM, (a) algorithm-level design, (b) macro-level design, (c) tile-level design, and (d) core-level design.

III. PAR-CIM Architecture

Overall Architecture. Fig. 3 depicts the overall architecture of PAR-CIM, designed to address the challenges discussed above. At the macro-level, the architecture comprises four reconfigurable tiles (R-Tiles), four cascaded reconfigurable shifters (R-Shifts), a global buffer, and additional peripheral circuits. At the tile-level, each R-Tile integrates two HP-Cores, two LA-Cores, an MMRC, and a local buffer. At the core-level, a LA-Core includes a 2Kb input buffer, a 2Kb 8T-SRAM weight storage array, 256 bitwise multipliers, LGAC-AT, an accumulator, and a set of read & write circuits. The HP-Core employs a precise AT (PAT), while other modules are identical to those in the LA-Core. At the algorithm-level, we propose FMBQ algorithm, validated on four traditional convolutional neural networks (CNNs) and two novel lightweight networks.

Main Features. We highlight three features in Fig. 3 using distinct colors. First, the LA-Core leverages LGAC-AT to construct SAAT, DAAT, and CAAT, providing configurable trade-offs among accuracy, power, and area. Notably, DAAT achieves full operation with only 0.35× of resource per bit, and supports MAC operations through integrated multipliers and accumulators (Feature 1). Furthermore, the FMBQ algorithm dynamically adjusts the quantization bit to achieve a balance between model size and accuracy. Combined with the HPLA scheme, this facilitates efficient multi-bit data computation (Feature 2). Last, the MBRM strategy orchestrates dataflow both within and between R-Tiles (Feature 3), enabling reconfigurable computing with minimal resource overhead, superior energy efficiency, and remarkable architectural flexibility.

A. *Approximate Optimization in Computing*

LGAC-AT Design Scheme. We implement layer/gate-level approximate optimizations in the ATs, which provide three selectable design options—SAAT, DAAT, and CAAT.

TABLE 2. Performance Comparison of Different ATs.

	RMSE	Transistor(T)	Power(uW)	Bits
PAT	0.00%	10486	37.46	256
SAAT	2.49%	5430(48.22%↓)	28.48(23.97%↓)	256
DAAT	4.51%	3630(65.38%↓)	20.62(44.95%↓)	256
CAAT	0.00%	8408(19.82%↓)	40.91(9.21%↑)	256

Both SAAT and DAAT employ layer-level optimization, replacing some layers of the PAT with OR and AND gates. In the PAT, the precise half adder (PHA) requires one XOR gate (8T) and one AND gate (6T), while a single OR or AND gate requires only 6T, reducing resource overhead but introducing errors. During computation, AND gates may cause a "*-1*" error, while OR gates may cause a "*+1*" error. Therefore, interleaving AND and OR gates can reduce errors [5]. As illustrated in Fig. 4(a), SAAT is derived from PAT by replacing its first layer with interleaved AND and OR gates. For a 16b input, SAAT consists of four layers, the first layer uses interleaving AND and OR gates, the second and third layers employ precise full adders (PFAs), and the fourth layer consists of PHAs, with the outputs defined as *O[4:1]* and *O[0]* set to zero. The output *O[4:0]* either matches the correct result or is evenly distributed around it, allowing errors to cancel out. Fig. 4(b) shows that DAAT modifies PAT by substituting its first two layers with interleaved AND and OR gates. Specifically, DAAT comprises four basic units: A^3, A^2O, O^2A, and O^3 (A = AND gate, O = OR gate), with each unit output assigned a weight of 2^2. Errors from these units are balanced when given the same input. For instance, with the input 4'b0001, A^3, A^2O, and O^2A introduce three "*-1*" errors, while O^3 introduces a "*+3*" error, resulting in error compensation.

CAAT utilizes gate-level optimization focusing on individual adder logic, as shown in Fig. 4(c). The XOR gate (8T) in PHA is replaced with an OR gate (6T), forming a

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.

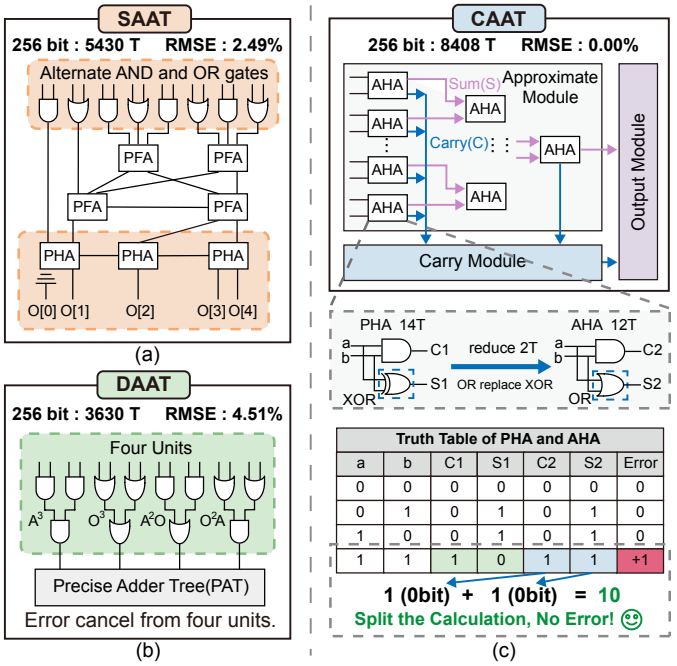


Fig. 4. LGAC-AT design: (a) SAAT, (b) DAAT, and (c) CAAT.

novel approximate half adder (AHA). Area analysis indicates that AHA requires only 12T, a reduction from PHA's 14T, leading to approximately 14% area savings for a single adder and approximately 19.82% for the complete AT array. AHA outputs "*11*" (with an error of "+*1*") only when both inputs are *1*. To address this error, we propose a carry-out computation mode that separates the AHA outputs *Carry(C)* and *Sum(S)*, each with a weight of 2^0. Thus, when both inputs are *1*, $C + S = 10$, effectively eliminating the error. In CAAT, each layer's *C* feeds into the carry module, which counts the number of *1* in *C* and passes the result to the output module. Simultaneously, each layer's *S* is passed to the next layer, iterating the process until precise results are achieved.

We model and validate four ATs using Python and Virtuoso, analyzing their RMSE, transistor number, and power. Detailed data are presented in Tab. 2. Both SAAT and DAAT reduce power and area with minimal computational errors. CAAT achieves error-free computation and saves 19.82% of area, with a slight but acceptable increase in power that does not impact the overall performance of PAR-CIM. To ensure accurate transistor number, we use both Python modeling and Virtuoso circuit implementations. For instance, the PAT requires 10486T (255 PHAs, 247 PFAs), while DAAT requires only 3630T (96 AND gates, 96 OR gates, 63 PHAs, 57 PFAs), resulting in an area reduction of 65.38%. By comparison, the SAAT demonstrates area reduction of 48.22%. Furthermore, subsequent verification confirms that three types of AATs result in less than 0.94% accuracy loss when processing INT1 DNNs. Considering the trade-offs between accuracy, power, and area, DAAT demonstrates the best overall performance.

Fractional Bitwidth Definition. To quantify the performance improvements, we define fractional bitwidth as follows:

$$Bit_{Fraction} = \frac{OptimizedResource/CalculatedBits}{InitialResource/CalculatedBits} \quad (1)$$

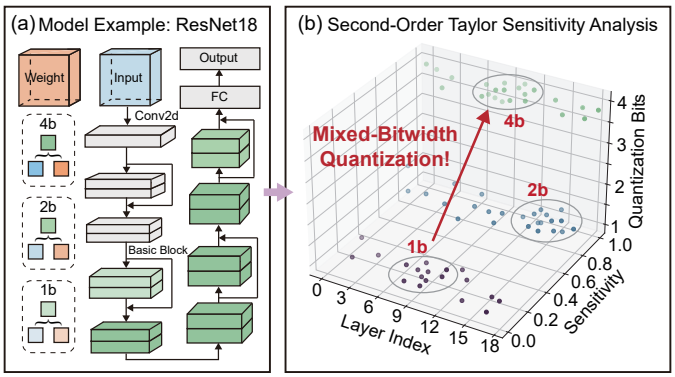


Fig. 5. Taylor sensitivity analysis on ResNet18.

According to Eq. (1), SAAT, DAAT, and CAAT achieve $Bit_{Fraction}$ of 0.52b, 0.35b, and 0.80b, respectively, significantly enhancing the bitwidth handling efficiency relative to resource overhead in this work. In addition, this work presents a 1/2/4b mixed-bitwidth quantization algorithm, FMBQ. This algorithm, leveraging the computational capacity expansion offered by the aforementioned AATs, achieves a fractional bit quantization range of 0.35b to 4b.

B. Quantization and Mapping in Data Storage

In pursuit of efficient neural network computation on hardware with limited resources, we present a novel mixed-bitwidth quantization and mapping scheme for data storage, FMBQ-HPLA. This scheme advances beyond traditional mixed-bitwidth quantization by enabling $Bit_{Fraction}$ through LGAC-AT, resulting in a marked improvement in resource utilization and computational flexibility.

Taylor Sensitivity Analysis. As shown in Fig. 5, using ResNet18 as an example, we conduct a second-order Taylor sensitivity analysis based on Eq. (2) and Eq. (3) to evaluate the quantization sensitivity of each layer in ResNet18. This analysis is performed to maximize performance while minimizing the accuracy loss.

$$L(\theta) \approx L(\theta_0) + \frac{1}{2}(\theta - \theta_0)^T H(\theta_0)(\theta - \theta_0) \quad (2)$$

$$Sensitivity = \|diag(H(\theta_0))\|_2 \quad (3)$$

In Eq. (2), the Hessian matrix $H(\theta_0)$ captures the second-order derivative information of the loss function concerning the parameters. Specifically, the diagonal elements of $H(\theta_0)$ represent the impact of individual parameter changes on the loss function, which indicates the sensitivity of the model to each parameter. Eq. (3) evaluates the overall sensitivity of a layer by calculating the L_2 norm of the diagonal elements of $H(\theta_0)$, providing a comprehensive measure of the model's response to combined parameter variations within that layer. A higher sensitivity value implies that the model is more vulnerable to quantization effects in the given layer, requiring a more refined quantization approach to avoid performance losses. Capitalizing on this predictive sensitivity analysis, we can adaptively assign varying quantization bitwidths across different layers, maximizing model compression while preserving accuracy.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.

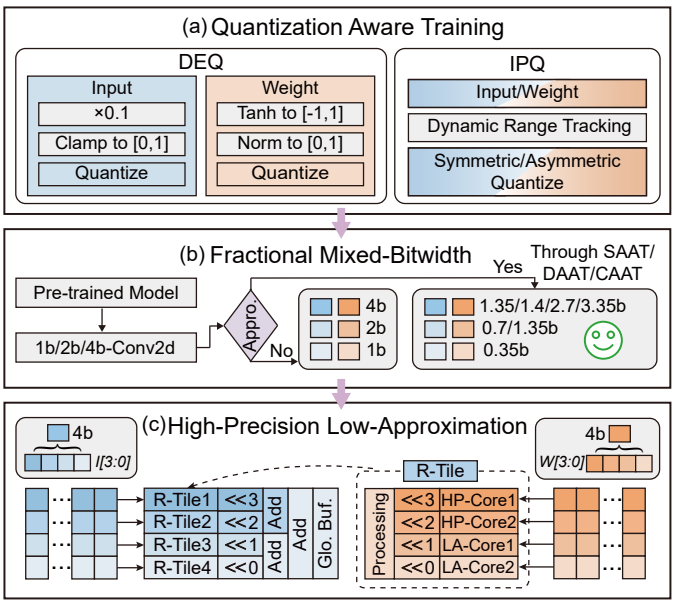


Fig. 6. Procedure of FMBQ algorithm.

Procedure of FMBQ Algorithm. Following the second-order Taylor sensitivity analysis, we will set the suitable quantization precision for the network according to its quantization sensitivity. The FMBQ algorithm, as illustrated in Fig. 6(a), adopts two quantization strategies. The first quantization scheme, DoReFa-Net-based quantization (DEQ), truncates inputs and weights to [0,1] and [-1,1], respectively, followed by multi-bit mapping. The second is intelligent perceptual quantization (IPQ), which can dynamically track activation and weight ranges according to layer-wise or channel-wise quantization requirements, calculating the quantization scale factor and zero-point, and supporting both symmetric and asymmetric quantization. While IPQ incurs higher computational overhead, it achieves superior accuracy. By integrating DEQ, IPQ, and Taylor sensitivity analysis, FMBQ facilitates arbitrary mixed-bitwidth quantization. In this work, 1/2/4b quantization is adopted to align with PAR-CIM. Applying FMBQ to ResNet18 results in 89.04% compression with only 0.60% accuracy loss relative to FP32, showing a well-balanced trade-off. As depicted in Fig. 6(b), pre-trained models after quantization aware training (QAT) can incorporate approximate functions to achieve low $Bit_{Fraction}$ further reducing resource overhead, which is a key advantage of FMBQ.

HPLA Mapping Scheme. As shown in Fig. 6(c), the HPLA scheme is employed for efficient data mapping and storage. The essential functionality of HPLA lies in dynamically selecting either the PAT or AAT to facilitate collaboration between the HP-Core and LA-Core. In light of the considerable influence of inaccuracies in high-bitwidth arithmetic, HPLA assures precision with high-bitwidth precise calculations and curtails overhead using low-bitwidth approximate calculations. This strategy effectively mitigates precision loss while optimizing resource utilization. Within one R-Tile, the most significant bit (MSB) and the second MSB of 4b weights, or the MSB of 2b weights, are stored in the HP-Core for

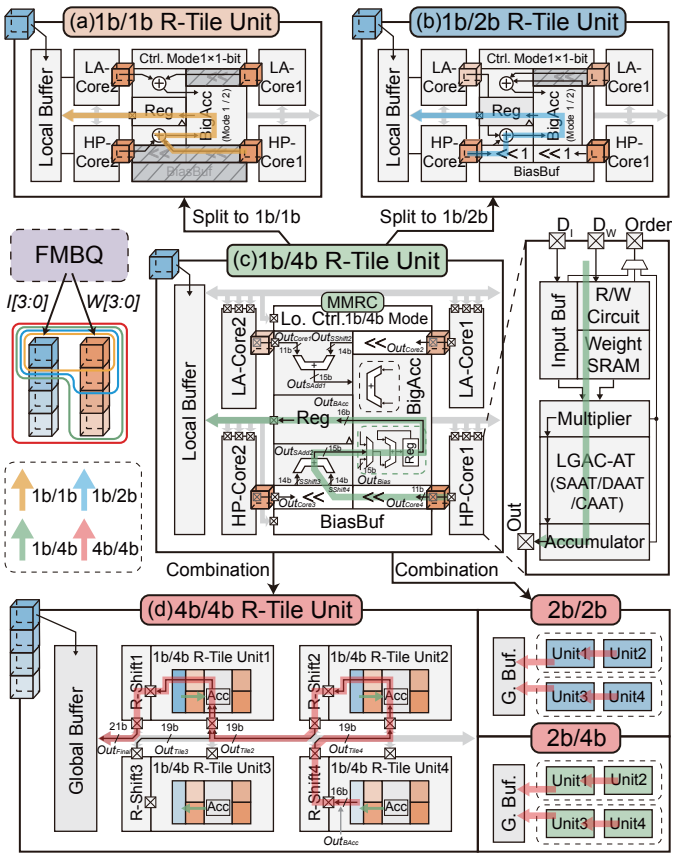


Fig. 7. MBRM design for different computation modes, (a) 1b/1b, (b) 1b/2b, (c) 1b/4b, and (d) m-b/n-b.

precise computation. The remaining lower bits are stored in the LA-Core for approximate computation, effectively reducing AAT-induced errors. For 4b or 2b weight computations, bit-wise shift operations are designed. In contrast, for 1b weights, there is an option to store them in HP-Core/LA-Core without requiring shift operations.

C. *Reconfigurable Design and Dataflow*

MBRM Design Strategy. We propose the MBRM strategy to optimize data flow within each R-Tile and between R-Tiles. As illustrated in Fig. 7(c), our approach introduces an MMRC composed of reconfigurable shifters, adders, and registers with low area and power overhead. The MMRC synergistically combines with HP-Core and LA-Core through high/low-bit partitioning, forming a unified R-Tile unit. By implementing the MBRM strategy to dynamically allocate computational resources for m-b/n-b operational modes, PAR-CIM achieves enhanced energy efficiency and architectural flexibility.

Reconfigurable Dataflow. As shown in Fig. 7(c), during 1b/4b computations within an R-Tile, the 4b weight *W[3:0]* is split into four 1b segments. The two MSBs (*W[3:2]*) are processed with inputs in HP-Cores, while the two least significant bits (LSBs, *W[1:0]*) are computed in LA-Cores. MMRC receives outputs $Out_{Corei}[10:0]$ from four cores and performs weight bit-dependent shift operations: the MSB *W[3]*, computed in HP-Core1, is shifted left by 3 bits, while the outputs from HP-Core2, LA-Core1, and LA-Core2 are

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.

shifted by 2, 1, and 0 bits. The shifted outputs $Out_{SShift}[13:0]$ are pairwise summed via upper and lower full adders to generate partial sums $Out_{SAddi}[14:0]$, which are then accumulated with $Out_{Bias}[14:0]$ in the big accumulator (BigAcc). The final result $Out_{BAcc}[15:0]$ is stored in the local buffer.

Similarly, each R-Tile can split to support 1b/1b and 1b/2b computation modes, as shown in Fig. 7(a) and Fig. 7(b). In 1b/1b computation mode, all shifters remain unshifted and only adders are performed to obtain correct MAC results. In 1b/2b computation mode, shifters are programmed to either shift left by one bit or remain unshifted. The HP-Core output is left-shifted by one bit while the LA-Core result remains unshifted, with both results subsequently summed through dedicated upper and lower adders.

Across multiple R-Tiles, as shown in Fig. 7(d). For 4b/4b operations, the input data $I[3:0]$ is split into four 1b values distributed across distinct R-Tiles, each R-Tile performs 1b/4b MAC operations with pre-stored weights $W[3:0]$. Outputs $Out_{BAcc}[15:0]$ stored in each R-Tile's local buffer are left-shifted in R-Shift units according to the positional weight of input $I[3:0]$, generating $Out_{Tilei}[18:0]$ for accumulation in the BigAcc. The final result $Out_{Final}[20:0]$ is stored in the global buffer. Furthermore, 1b/2b and 1b/4b modes can be combined to enable 2b/2b and 2b/4b computing modes.

In PAR-CIM, each DCIM core is restricted to performing 1b/1b computations, with the MBRM strategy exclusively enabling multi-bit computations. This fine-grained reconfigurable approach effectively addresses challenges such as high latency and resource underutilization in multi-bit data processing, thereby achieving 1/2/4b highly flexible computation and energy efficiency of up to 3048 TOPS/W.

IV. Evaluation and Discussion

PAR-CIM achieves significant performance improvements through the co-optimization of circuit, algorithm, and architecture. This section provides a comprehensive experimental analysis of PAR-CIM, focusing on its hardware efficiency, algorithmic superiority, and overall system performance.

A. Experimental Setup

Hardware Implementation. PAR-CIM is evaluated on 40nm technology and includes a global buffer, a single instruction multiple data (SIMD) module, a top-level control module, and four R-Tiles. Each R-Tile contains four DCIM cores, with each core comprising a 2Kb input buffer and a 2Kb SRAM array. Key custom circuits, including input buffers, SRAM arrays, different types of ATs, and multipliers, are designed using Cadence Virtuoso. Input buffer and SRAM array undergo area and power evaluation with Cadence Spectre. ATs and multipliers are analyzed for power using Cadence Spectre and for area using Design Compiler. The remaining digital modules are implemented in Verilog HDL and synthesized with Design Compiler. Fig. 8 illustrates the PAR-CIM macro layout, where the physical implementation is realized through Cadence Innovus place-and-route flow. The overall area of R-Tile modules is evaluated using post-layout extraction techniques.

TABLE 3. Efficiency of Different AAT-Cores.

Type of AAT-Cores	PAT	SAAT	DAAT	CAAT
Energy Efficiency (TOPS/W)	3121	3996	**5296**	2879
Area Efficiency (TOPS/mm^2)	6.95	7.69	**7.90**	7.17

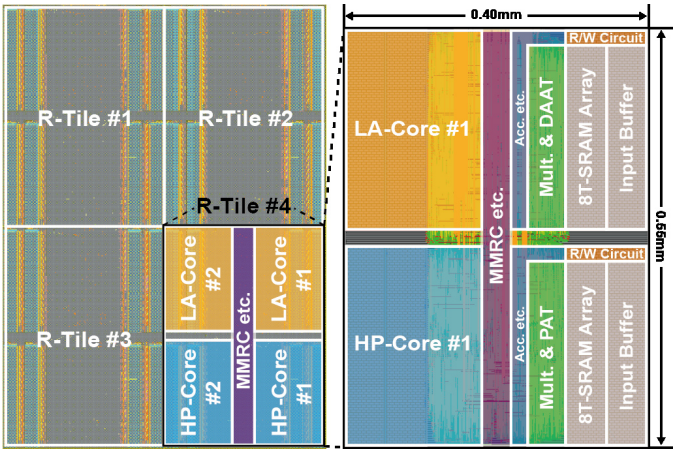


Fig. 8: PAR-CIM macro layout in 40nm technology.

Algorithm Settings. The models used in this work are trained on the CIFAR-10 dataset using a GeForce RTX 4090D GPU with CUDA 11.8 and PyTorch 2.0.0 on Ubuntu 20.04 LTS. The initial learning rate is 0.01, and the batch size is set to 256. The SGD optimizer is used with momentum as 0.9 and a weight decaying coefficient of *5e-4* for model training. Furthermore, a ReduceLROnPlateau learning rate scheduler is employed, halving the learning rate when the inference performance shows no improvement for 10 epochs. The proposed FMBQ algorithm supports arbitrary mixed-bitwidth quantization. In this work, we primarily investigate 1/2/4b quantization to match hardware constraints. Additionally, pre-trained QAT models are integrated into approximate functions to facilitate equivalent-condition modeling.

B. Efficiency Evaluation of Hardware

To comprehensively evaluate the DCIM cores (PAT-Core, SAAT-Core, DAAT-Core, and CAAT-Core) designed using PAT, SAAT, DAAT, and CAAT, we analyze their energy and area efficiency. Tab. 3 presents a detailed comparison of their respective performance metrics. The results demonstrate that both SAAT-Core and DAAT-Core significantly outperform PAT-Core. Specifically, DAAT-Core achieves 5296 TOPS/W and 7.90 TOPS/mm^2, showcasing its superior performance in both aspects. CAAT-Core achieves a 3.17% improvement in area efficiency while maintaining accuracy. However, due to its complex computational flow, its energy efficiency is slightly lower than PAT-Core, though still within an acceptable range. Considering the performance trade-offs among the three novel DCIM cores, DAAT demonstrates the best overall performance. Therefore, subsequent hardware experiments will primarily be based on DAAT.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.

TABLE 4. Efficiency of Different AT Types and Modes.

AT Types	Efficiency	Computation modes					
		1b/1b	1b/2b	1b/4b	2b/2b	2b/4b	4b/4b
PAT-Type	TOPS/W	2539	1239	612	586	288	148
	TOPS/mm^2	6.88	3.43	1.71	1.71	0.85	0.43
SAAT-Type	TOPS/W	2787	1357	670	638	313	161
	TOPS/mm^2	7.22	3.60	1.80	1.79	0.90	0.45
DAAT-Type	**TOPS/W**	**3048**	**1480**	**730**	**693**	**339**	**175**
	TOPS/mm^2	**7.32**	**3.65**	**1.82**	**1.82**	**0.91**	**0.45**
CAAT-Type	TOPS/W	2455	1199	593	568	279	143
	TOPS/mm^2	6.99	3.49	1.74	1.74	0.87	0.43

Beyond evaluating different types of DCIM cores, we also comprehensively assess the performance of the PAR-CIM architecture. Our proposed MBRM enhances architectural flexibility by controlling dataflow within and between R-Tiles, thereby supporting six computing modes: 1b/1b, 1b/2b, 1b/4b, 2b/2b, 2b/4b, and 4b/4b. Each R-Tile consists of two HP-Cores and two LA-Cores, with the LA-Cores capable of selecting different AT types as needed. We summarize the energy and area efficiency of PAR-CIM across different computation modes and AT types configurations as shown in Tab. 4. PAR-CIM demonstrates exceptional efficiency under various settings. Furthermore, integrating SAAT, DAAT, and CAAT significantly enhances the performance of PAR-CIM. Notably, with the DAAT-Type configuration, it achieves energy efficiency ranging from 175 to 3048 TOPS/W and area efficiency from 0.45 to 7.32 TOPS/mm^2, representing up to a 20.05% improvement and a 6.40% improvement compared to PAT-Type, respectively. These results underscore the versatility and effectiveness of PAR-CIM in reducing resource overhead, enhancing energy efficiency, and increasing architectural flexibility. The broad configurability and substantial performance gains validate the proposed approach as a highly adaptable and efficient solution for diverse computational requirements.

C. *FMBQ Evaluation on PAR-CIM*

To evaluate the generality of FMBQ on PAR-CIM, we deploy various neural networks on CIFAR-10. Traditional CNNs include LeNet [20], AlexNet [21], VGG16 [22], and ResNet18, while lightweight networks feature MobileNetV3 (small) [23] and a custom V-FuseMBA based on the Mamba module. The Mamba module combines the strengths of recurrent neural networks (RNNs), Transformers, and state space models to achieve an efficient RNN-like decoding pattern, maintaining constant computational and memory complexity during inference [24]. Mamba excels at capturing complex dependencies in sequential data, but it is less effective for non-sequential data like images. To address this limitation, we incorporate the Mamba block, basic block, squeeze-and-excitation (SE), and inception block to enhance feature extraction for vision tasks. Demonstrating FMBQ's potential in emerging network architectures, the optimized V-FuseMBA achieves a size of just 4.17MB and a Top-1 accuracy of 93.60%.

Fig. 9 shows the Top-1 accuracy and normalized model size for six networks under INT1, INT2, INT4, FMBQ, and FP32. As shown in Fig. 9(a), the accuracy loss of FMBQ compared to FP32 is below 0.84% across networks, while outperforming INT4 in accuracy. Fig. 9(b) highlights that FMBQ achieves compression rates of up to 93.44%, outperforming INT4 as well. Notably, ResNet18 and V-FuseMBA based on the FMBQ achieve accuracies of 93.67% and 92.86%, compress models by 89.04% and 86.61%, with accuracy loss below 0.74%. This demonstrates the effectiveness of FMBQ across networks with minor accuracy loss, ensuring its applicability in PAR-CIM.

Algorithm 1: Approximate Network Modeling (ANM)

Input : Approximate Convolution Input
$I[\text{Batch}_{size}][\text{Height}_{in}][\text{Width}_{in}][\text{Channel}_{in}]$
$W[\text{Kernel}_{height}][\text{Kernel}_{width}][\text{Channel}_{in}][\text{Channel}_{out}]$
Output: Approximate Convolution Output
$O[\text{Batch}_{size}][\text{Height}_{out}][\text{Width}_{out}][\text{Channel}_{out}]$
Define : $\text{Conv2d}_{app}(I, W, O, AT_{type}, Quant_{bit})$, which is illustrated below

Determine INT1 or INT4 quantization;
if $Quant_{bit} == INT1$ **then** // $Conv2d^{int1}_{app}$
  $Binarized(I, W)$;
  $pad \leftarrow 1$ *then* $I_{pad} \leftarrow F.pad(I, pad)$;
  $(I_{unfold}, W_{unfold}) \leftarrow Unfold(I_{pad}, W)$;
  $Mul \leftarrow I_{unfold} \times W_{unfold}$; // $Matmul_{app}$
  $Mul_{trans} \leftarrow \frac{Mul-1}{-2}$;
  $O_{unfold} \leftarrow AAT(Mul_{trans}, AT_{type})$; // $== Feature\ 1$
  $O^{trans}_{unfold} \leftarrow O_{unfold} \times (-2) + 1$;
  $O \leftarrow Fold(O^{trans}_{unfold})$;
end
else if $Quant_{bit} == INT4$ **then** // $Conv2d^{int4}_{app}$
  $Quantize_to_INT4(I, W)$;
  for *the pos of bit* $\leftarrow 0$ **to** 3 **do** // $HPLA$
    $Conv2d_{app}(I_{bit_{pos}}, W_{bit_{pos}}, O_{bit_{pos}}, AT_{type}, INT1)$;
  end
  $O \leftarrow O_{bit_3} \times 2^3 + O_{bit_2} \times 2^2 + O_{bit_1} \times 2^1 + O_{bit_0} \times 2^0$;
end

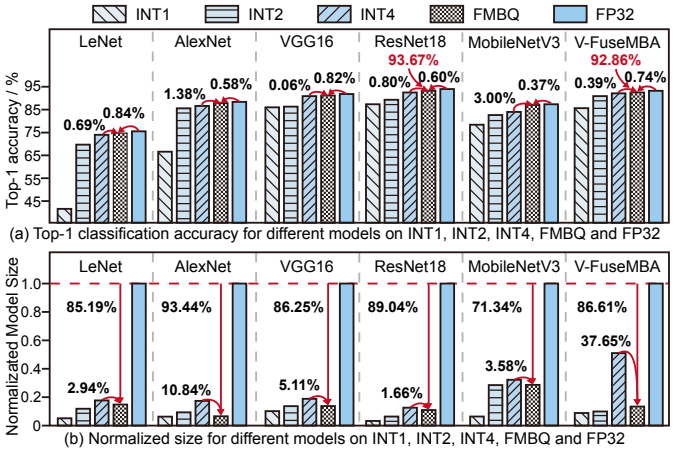


Fig. 9. Top-1 accuracy and normalized size for different models on INT1, INT2, INT4, FMBQ and FP32.

To further validate the effectiveness of HPLA, we select two representative networks, ResNet18 and V-FuseMBA, and incorporate the three AAT-based (Fig. 4) approximate convolution functions. To effectively simulate hardware computation results, we implement an equivalent model of PAR-CIM using Python and design an approximate network modeling (ANM) algorithm to ensure the consistency between network computation results and chip outputs. The ANM algorithm's process is detailed in Alg. 1.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.

TABLE 5. Detailed Comparisons with Prior Works.

ADCIM ← → RDCIM ← → DCIM

	ISSCC'22 [5]		PAR-CIM (This Work)			TCAS-I'24 [9]	VLSI'22 [27]	ISSCC'22 [28]	DAC'24 [25]	DAC'23 [26]
	DIMC-S	DIMC-D								
Techology	28nm	28nm	**40nm**			55nm	12nm	28nm	28nm	28nm
MAC Operation	Digital	Digital	**Digital**			Digital	Digital	Digital	Digital	Digital
Array Size	16Kb	16Kb	**64Kb**			4Kb	8Kb	32Kb	32KB	8KB
Cell Type	6T SRAM	6T SRAM	**8T SRAM**			6T SRAM	6T SRAM	6T SRAM	6T SRAM	8T SRAM
Power Supply(V)	0.45-1.10	0.45-1.10	**0.90**			0.70-1.20	0.72	0.80	0.72-0.90	-
Operating Frequency(MHz)	250	280	**250**			100	-	333	500	783
Input Toggle Rate	25%	25%	**10%**			10%	-	-	-	-
RMSE	4.03%	6.76%	**2.49%**[*1]	**4.51%**[*2]	**Precise**[*3]	-	-	-	-	-
Input	1-4b	1b	**1-4b**			1-8b	4-8b	1-8b	8b	8b
Weight	1b	1b	**1-4b**			1-8b	4b/8b	1b/4b/8b	8b	8b
Energy Efficiency (TOPS/W)(1b/1b)	992[*4]	2219	**2787**[*1]	**3048**[*2]	**2455**[*3]	2238	1939.2[*4]	1752.32[*4]	2892.8[*4]	1891.8[*4]
CIFAR-10 Accuracy (1b/1b)	89.00%[*5]	86.90%[*5]	**87.19%**[*1] **(ResNet18)**	**86.93%**[*2] **(ResNet18)**	**87.52%**[*3] **(ResNet18)**	82.45% (ResNet18)	-	-	-	-

Lower RMSE → Higher Energy Efficiency

[*1] Measured at SAAT-Type. [*2] Measured at DAAT-Type. [*3] Measured at CAAT-Type. [*4] Normalized to 1b/1b. [*5] No model information available.

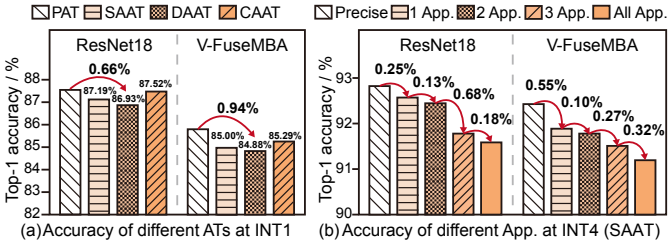


Fig. 10. Top-1 accuracy of different ATs and Approximation

Specifically, ANM first selects INT1 or INT4 quantization. In INT1 quantization, traditional convolution is replaced by an approximate convolution function ($Conv2d_{app}^{int1}$) using PAT, SAAT, DAAT, or CAAT equivalent adder functions. In INT4 quantization, the inputs/weights of convolutions are quantized to 4b ($Conv2d_{app}^{int4}$), each bit computed with a $Conv2d_{app}^{int1}$, followed by MAC operations. Similarly, network accuracy can be verified at different approximation levels by each $Conv2d_{app}^{int1}$ through selected ATs. In summary, ANM achieves chip-equivalent modeling via $Conv2d_{app}$ and validates HPLA's effectiveness via approximate network accuracy. As shown in Fig. 10, under INT1 quantization, the accuracy loss for both networks with SAAT, DAAT, and CAAT stays below 0.94%. Under SAAT, DAAT, and CAAT, ResNet achieves accuracies of 87.19%, 86.93%, and 87.52%, respectively. For INT4 quantization, using SAAT, the classification accuracy improves as the approximation level decreases. The experimental results show that AATs have low accuracy loss. HPLA reduces errors further and boosts resource efficiency.

D. Comparison with Prior Works

Tab. 5 presents a comprehensive comparison of this work with prior DCIM designs. Firstly, regarding traditional DCIM, PAR-CIM achieves 1.05× and 1.61× energy efficiency of [25] [26]. Additionally, PAR-CIM seamlessly integrates the advantages of ADCIM and RDCIM, delivering superior energy efficiency, architectural flexibility, and accuracy. Compared to prior ADCIM designs, PAR-CIM demonstrates higher energy efficiency with reduced RMSE. For instance, DAAT-Type demonstrates superior performance with its RMSE being lower than that of DIMC-D, while achieving energy efficiency of 3.07× and 1.37× that of [5], respectively. Notably, SAAT-Type and CAAT-Type attain energy efficiency up to 2.81× and 2.47× of [5], yet maintain low RMSE at 2.49% and precise value respectively, demonstrating exceptional precision-efficiency trade-off capabilities. In comparison with existing RDCIM designs, PAR-CIM enhances energy efficiency with minimal accuracy degradation while retaining a high degree of architectural flexibility. For example, at the RMSE of 4.51%, PAR-CIM achieves energy efficiency of 1.36×, 1.57×, and 1.74× those of [9], [27], and [28], respectively. Overall, PAR-CIM achieves peak 3048 TOPS/W at 1b/1b MAC operations, alongside an area efficiency of 7.32 TOPS/mm^2, surpassing most state-of-the-art designs.

V. Conclusion

In this work, we propose PAR-CIM, an energy-efficient and reconfigurable DCIM macro that combines the strengths of ADCIM and RDCIM, supporting six computation modes and achieving 3048 TOPS/W and 7.32 TOPS/mm^2 at 1b/1b MAC operations. At the circuit level, we leverage three types of AATs to achieve a flexible trade-off among area, power, and accuracy. At the algorithm level, the proposed FMBQ algorithm supports fractional bit processing, ensuring high effectiveness and adaptability. At the architecture level, the reconfigurable architecture enhances the hardware flexibility of PAR-CIM and complements the circuit and algorithm-level designs. Comprehensive experimental results demonstrate that PAR-CIM provides a promising path for neural network compression and acceleration. Despite the significant progress made by PAR-CIM, challenges remain. Future work will focus on improving area efficiency, optimizing hardware-software co-design, and building a more complete system platform to accommodate a wider range of neural network applications.

Acknowledgment

This work is supported partly by National Key Research and Development Program of China (Grant No. 2021YFB3601304, 2021YFB3601300), partly by Beijing Nova Program.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] X. Lin, S. Huang, and H. Jiang, "Voxel-cim: An efficient compute-in-memory accelerator for voxel-based point cloud neural networks," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025, pp. 1–9.

[2] W. Yi, K. Mo, W. Wang *et al.*, "Rdcim: Risc-v supported full-digital computing-in-memory processor with high energy efficiency and low area overhead," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 4, pp. 1719–1732, 2024.

[3] J.-S. Seo, "Advances and trends on on-chip compute-in-memory macros and accelerators," in *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.

[4] F. Tu, Z. Wu, Y. Wang *et al.*, "Trancim: Full-digital bitline-transpose cim-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 6, pp. 1798–1809, 2023.

[5] D. Wang, C.-T. Lin, G. K. Chen *et al.*, "Dimc: 2219tops/w 2569f^2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 266–268.

[6] K. Li, H. Huang, M. Huang *et al.*, "A 29.12 tops/w and 1.13 tops/mm^2 nas-optimized mixed-precision dnn accelerator with vector split- and-combination systolic in 28nm cmos," in *2024 IEEE Custom Integrated Circuits Conference (CICC)*, 2024, pp. 1–2.

[7] X. Zhang, V. Sharma, Y. Lu *et al.*, "A 400mhz 249.1tops/w 64kb fully-reconfigurable sram-based digital compute-in-memory macro for accelerating cnns," in *2023 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, 2023, pp. 1–3.

[8] H. Kim, T. Yoo, T. T.-H. Kim *et al.*, "Colonnade: A reconfigurable sram-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, 2021.

[9] H. You, W. Li, D. Shang *et al.*, "A 1–8b reconfigurable digital sram compute-in-memory macro for processing neural networks," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 4, pp. 1602–1614, 2024.

[10] S. Zhou, Z. Ni, X. Zhou *et al.*, "Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients," *arXiv preprint arXiv:1606.06160*, 2016.

[11] K. He, X. Zhang, S. Ren *et al.*, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[12] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[13] Y. Qiu, Y. Ma, W. Zhao *et al.*, "Dcim-gcn: Digital computing-in-memory to efficiently accelerate graph convolutional networks," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2022.

[14] S.-E. Hsieh, C.-H. Wei, C.-X. Xue *et al.*, "7.6 a 70.85-86.27tops/w pvt-insensitive 8b word-wise acim with post-processing relaxation," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 136–138.

[15] Y. He, H. Diao, C. Tang *et al.*, "7.3 a 28nm 38-to-102-tops/w 8b multiply-less approximate digital sram compute-in-memory macro for neural-network inference," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 130–132.

[16] A. Guo, X. Chen, F. Dong *et al.*, "34.3 a 22nm 64kb lightning-like hybrid computing-in-memory macro with a compressed adder tree and analog-storage quantizers for transformer and cnns," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, 2024, pp. 570–572.

[17] H. Jia, M. Ozatay, Y. Tang *et al.*, "15.1 a programmable neural-network inference accelerator based on scalable in-memory computing," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 236–238.

[18] F. Tu, Y. Wang, Z. Wu *et al.*, "A 28nm 29.2tflops/w bf16 and 36.5tops/w int8 reconfigurable digital cim processor with unified fp/int pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.

[19] V. Sharma, H. Kim, and T. T.-H. Kim, "A 64 kb reconfigurable full-precision digital reram-based compute-in-memory for artificial intelligence applications," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 8, pp. 3284–3296, 2022.

[20] Y. Lecun, L. Bottou, Y. Bengio *et al.*, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[21] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.

[22] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[23] A. Howard, M. Sandler, B. Chen *et al.*, "Searching for mobilenetv3," in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[24] L. Guo, Z. Zhu, T. Liu *et al.*, "Towards floating point-based attention-free llm: Hybrid pim with non-uniform data format and reduced multiplications," in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024, pp. 1–9.

[25] C. Duan, J. Yang, Y. Wang *et al.*, "Towards efficient sram-pim architecture design by exploiting unstructured bit-level sparsity," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024, pp. 1–6.

[26] A. Sridharan, F. Zhang, Y. Sui *et al.*, "Dspimm: A fully digital sparse in-memory matrix vector multiplier for communication applications," in *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.

[27] C.-F. Lee, C.-H. Lu, C.-E. Lee *et al.*, "A 12nm 121-tops/w 41.6-tops/mm^2 all digital full precision sram-based compute-in-memory with configurable bit-width for ai edge applications," in *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, 2022, pp. 24–25.

[28] B. Yan, J.-L. Hsu, P.-C. Yu *et al.*, "A 1.041-mb/mm^2 27.38-tops/w signed-int8 dynamic-logic-based adc-less sram compute-in-memory macro in 28nm with reconfigurable bitwise operation for ai and embedded applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 188–190.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:41 UTC from IEEE Xplore. Restrictions apply.